

Variational Autoencoders for Telugu Character Generation

A Comparative Study Across Two Professional Fonts

PAMIDI ROHIT
Roll: S20230030398

IIITS
Department of AI & DS

February 2, 2026

Why Telugu Character Generation?

- Telugu is spoken by 74 million people
- Complex script with rounded, curved strokes
- Limited training data for machine learning models
- Need synthetic data for OCR and recognition systems

Why VAEs?

- Stable training compared to GANs
- Learn meaningful patterns in data
- Can generate new, realistic characters
- Useful for data augmentation

Research Gap

- No studies comparing VAEs on different Telugu fonts
- Unclear which font characteristics help VAE learning
- Missing guidelines for Indic script generation

Our Contribution:

- First systematic comparison of two Telugu fonts
- Identify what makes fonts easier or harder to learn

13 Telugu Vowels (Achulu)

Code	Transliteration	Name
a	a	short a
aa	ā	long a
i	i	short i
ii	ī	long i
u	u	short u
uu	ū	long u
ri	ṛ	vocalic r
e	e	e
ai	ai	ai
o	o	o
au	au	au
am	aṁ	anusvara
ah	aḥ	visarga

Two Professional Fonts

1 Pothana2000

- Traditional style
- Complex stroke patterns
- 1,040 samples

2 Lohit Telugu

- Modern, clean design
- Simpler strokes
- 1,040 samples

Key Question

Which font works better with VAEs and why?

Dataset Statistics

Metric	Value
Total Samples	2,080
Unique Characters	13 vowels
Fonts Used	2
Font Sizes	4 (10, 14, 18, 22pt)
Samples per Character	160
Samples per Font	1,040
Image Resolution	128×128 pixels
Image Quality	300 DPI
Format	PNG grayscale
Augmentation Types	222 variations

Data Augmentation

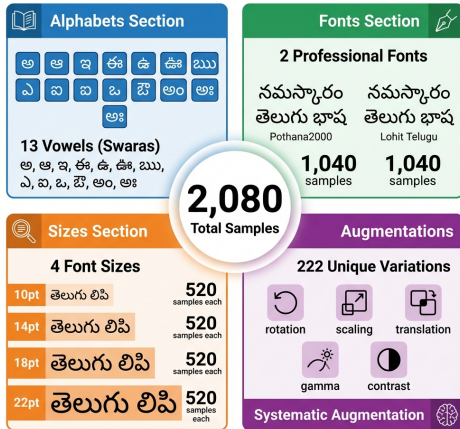
- **Rotation:** Small angle variations
- **Scaling:** Size adjustments
- **Translation:** Position shifts
- **Gamma:** Brightness changes
- **Contrast:** Intensity variations

Data Split

- Training: 70%
- Validation: 15%
- Testing: 15%

Dataset Overview Visualization

Dataset Overview: 2,080 Telugu Character Samples



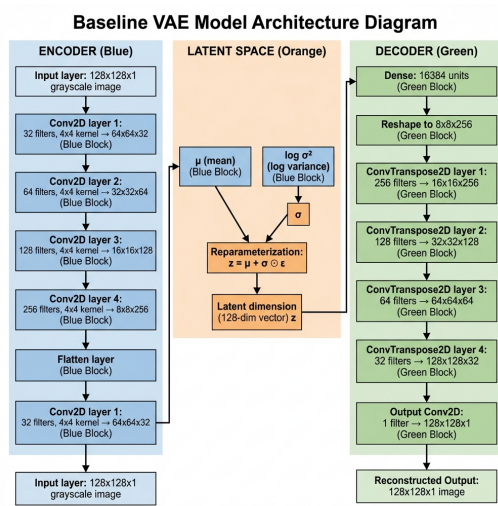
Three Model Architectures

Model	Key Feature	Purpose
1. Baseline VAE	Standard architecture 128-dim latent space	Establish baseline performance See which font is easier to learn
2. Beta-VAE	Controlled disentanglement Beta parameter ($\beta = 1, 2, 4, 10$) KL annealing schedule	Improve latent space quality Separate different character features Better generation control
3. Refined VAE	Residual connections Attention mechanism Multi-scale processing	Address observed limitations Learn from Experiments 1 & 2 Improve performance on both fonts

Progressive Approach

Start simple (Baseline) → Add disentanglement (Beta-VAE) → Add improvements based on what we learned (Refined VAE)

Model 1: Baseline VAE Architecture



Model 1: Baseline VAE - Technical Details

Encoder (Blue)

- Input: 128×128 grayscale image
- Conv Layer 1: 32 filters $\rightarrow 64 \times 64 \times 32$
- Conv Layer 2: 64 filters $\rightarrow 32 \times 32 \times 64$
- Conv Layer 3: 128 filters $\rightarrow 16 \times 16 \times 128$
- Conv Layer 4: 256 filters $\rightarrow 8 \times 8 \times 256$
- Flatten layer
- Output: Mean (μ) and Variance ($\log \sigma^2$)

Latent Space (Orange)

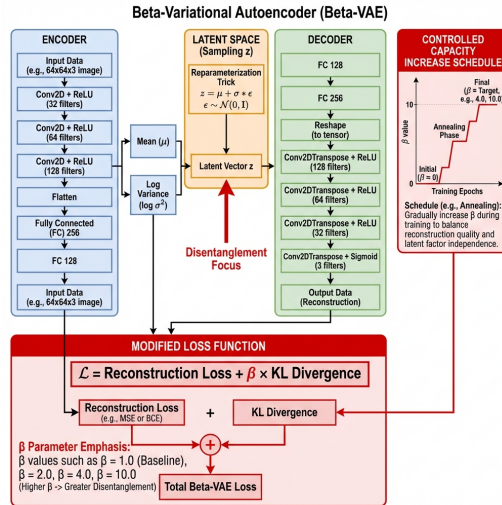
- Reparameterization: $z = \mu + \sigma \odot \epsilon$
- Latent dimension: 128-dim vector

Decoder (Green)

- Dense layer: 16,384 units
- Reshape to $8 \times 8 \times 256$
- ConvTranspose 1: 256 filters $\rightarrow 16 \times 16 \times 256$
- ConvTranspose 2: 128 filters $\rightarrow 32 \times 32 \times 128$
- ConvTranspose 3: 64 filters $\rightarrow 64 \times 64 \times 64$
- ConvTranspose 4: 32 filters $\rightarrow 128 \times 128 \times 32$
- Output Conv: 1 filter $\rightarrow 128 \times 128 \times 1$
- Final: Reconstructed image

Loss Function:

Model 2: Beta-VAE Architecture



Model 2: Beta-VAE - Technical Details

Key Innovation: Beta Parameter

The loss function becomes:

$$\mathcal{L} = \text{Reconstruction Loss} + \beta \times \text{KL Divergence}$$

What Beta Does:

- $\beta = 1.0$: Standard VAE (baseline)
- $\beta = 2.0$: Mild disentanglement
- $\beta = 4.0$: Stronger disentanglement
- $\beta = 10.0$: Maximum disentanglement

Trade-off

Higher β = Better disentanglement but potentially blurrier images

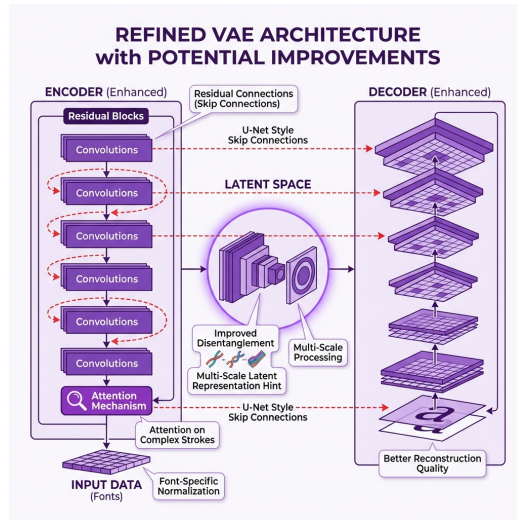
KL Annealing Schedule

- Start with $\beta = 0$ (focus on reconstruction)
- Gradually increase β during training
- Prevents posterior collapse
- Helps stable training

Benefits for Telugu Characters:

- Separate stroke patterns from character identity
- Learn independent features (curves, loops, angles)
- Better control over generation
- More interpretable latent space

Model 3: Refined VAE Architecture



Model 3: Refined VAE - Technical Details

Architectural Enhancements

① Residual Connections

- Skip connections like U-Net
- Preserve fine details
- Better gradient flow

② Attention Mechanism

- Focus on complex stroke patterns
- Handle intricate character features
- Improve reconstruction quality

③ Multi-Scale Processing

- Capture both details and structure
- Process at different resolutions
- Better for Telugu's curves and loops

Additional Features

④ Font-Specific Normalization

- Adapt to each font's characteristics
- Use font-specific statistics
- Improve training stability

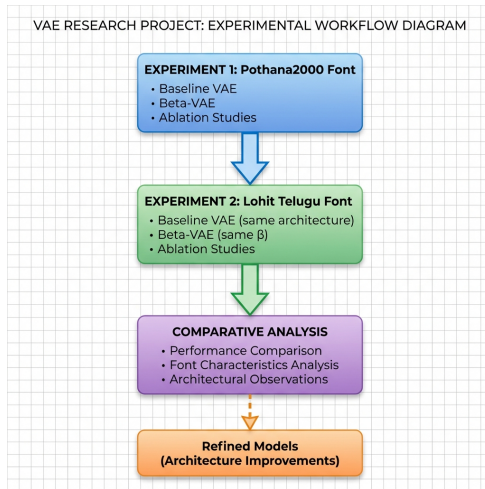
⑤ Modified Loss Weighting

- Adjust for harder fonts
- Balance reconstruction vs regularization
- Font-aware training

Goal

Apply lessons learned from Baseline and Beta-VAE experiments to create the best model for both fonts

Experimental Workflow



Experimental Design & Strategy

Two-Font Comparison Strategy

Experiment 1: Pothana2000 Font

- Baseline VAE
- Beta-VAE (multiple β values)
- Ablation studies

Experiment 2: Lohit Telugu Font

- Same architecture as Experiment 1
- Same hyperparameters
- Same evaluation protocol

Why This Design?

- Fair comparison (only font changes)

Controlled Variables

- Same architecture design
- Same hyperparameters
- Same optimization strategy
- Same loss functions
- Same evaluation metrics
- Same random seeds

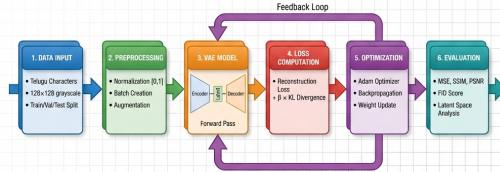
Variable Factor

- Font only (Pothana2000 vs Lohit Telugu)

Goal

Isolate font effect from all other factors for fair comparison

VAE TRAINING PIPELINE: Telugu Character Generation



Training Pipeline - Detailed Steps

Pipeline Steps

1 Data Input

- Telugu characters, 128×128 grayscale
- Train/Val/Test split (70/15/15)

2 Preprocessing

- Normalize to [0,1]
- Create batches
- Apply augmentation

3 VAE Model

- Encoder → Latent → Decoder
- Forward pass through network

4 Loss Computation

- Reconstruction loss (MSE or BCE)
- KL divergence loss
- Total loss = Recon + $\beta \times$ KL

5 Optimization

- Adam optimizer
- Backpropagation
- Weight updates

6 Evaluation

- MSE, SSIM, PSNR metrics
- FID score
- Latent space analysis

Feedback Loop

Continuous monitoring and adjustment throughout training

Reconstruction Quality Metrics

① Mean Squared Error (MSE) ↓

- Pixel-level accuracy
- Lower is better
- Range: $[0, \infty)$

② Structural Similarity (SSIM) ↑

- Considers structure and patterns
- Higher is better
- Range: $[-1, 1]$, ideal = 1

③ Peak Signal-to-Noise Ratio (PSNR) ↑

- Image quality measure
- Higher is better
- Typical range: 20-40 dB

Generation Quality Metrics

④ Fréchet Inception Distance (FID) ↓

- Compares real vs generated distributions
- Lower is better
- Good: ≤ 50 , Excellent: ≤ 10

Latent Space Quality

⑤ Mutual Information Gap (MIG) ↑

- Measures disentanglement
- Range: $[0, 1]$

⑥ Separated Attribute Predictability (SAP) ↑

- How well features are separated
- Range: $[0, 1]$

Expected Outcomes & Impact

Research Contributions

① First Comparative Study

- First systematic VAE comparison for Telugu fonts
- Benchmark for future research

② Font Performance Analysis

- Identify which font trains better
- Understand why certain fonts work better

③ Architectural Insights

- Observation-driven improvements
- Guidelines for Indic script generation

④ Public Release

- Open-source dataset
- Code and trained models

Practical Applications

• OCR Training Data

- Generate synthetic characters
- Improve recognition systems

• Digital Typography

- Automated font design
- Font style transfer

• Data Augmentation

- Create variations for training
- Address data scarcity

• Language Preservation

- Digital archiving of traditional fonts
- Support for Indic languages

Thank You!

Questions & Discussion

PAMIDI ROHIT

Roll: S20230030398

IIITS, Department of AI & DS

Variational Autoencoders for Telugu Character Generation